Brief Announcement: On Using Null Messages in a Byzantine Setting

Guy Goren
Yoram Moses
sgoren@campus.technion.ac.il
moses@technion.ac.il
The Viterbi Faculty of Electrical Engineering, Technion

ABSTRACT

In reliable settings, null messages allow the transfer of information without explicit communication in cases of interest. We investigate the use of null messages in the much more challenging Byzantine model (without signatures). Different ways of using null messages are discussed. One of them, called a *silent validation round*, can provide processes with global information about all correct sites of the system, without any message exchange. As a case study, we consider optimizing the behavior in failure-free runs of protocols for the classic Byzantine Consensus problem.

CCS CONCEPTS

•Computing methodologies →Distributed computing methodologies; •Theory of computation →Distributed algorithms; Algorithm design techniques;

KEYWORDS

Null messages, fault-tolerance, Byzantine Consensus

ACM Reference format:
Guy Goren and Yoram Moses. 2020. Brief Announcement: On Using Null Messages in a Byzantine Setting. In *Proceedings of ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, August 3–7, 2020 (PODC '20),* 3 pages.
DOI: 10.1145/3382734.3405717

1 USING NULL MESSAGES AND SILENT VALIDATION ROUNDS

Sending a null message is, in general, considerably cheaper than sending an explicit message. But what information can a null message convey? In a reliable synchronous model, a process j that does not receive a message from i is guaranteed that i purposely sent j a null message. If i's protocol is such that a null message would be sent only if some condition holds, then j is informed by i's null message that the condition holds. The possibility of failures changes the picture somewhat, since a null message can result from the fact that i is faulty, regardless of what its protocol specifies. Consequently, the only information that j learns when i's null message arrives is that *if i is correct* then the condition holds. In the crash failure model, explicit (non-null) messages are only ever sent according to the protocol. This makes null messages qualitatively different from explicit messages in the crash model, as discussed by [8]. In the Byzantine model, however, faulty processes may send arbitrary messages. Thus, in this model the only information that j learns when it receives an explicit message μ from i is that *if i is correct* then the conditions under which the protocol specifies that i should send μ holds. Essentially, in the Byzantine model the status of a null message is similar to that of an explicit message.[1] This is not because null messages are promoted in the Byzantine case, but rather because the information content of explicit messages is demoted there.

A trivial way of using null messages is by having them encode an *a priori* fixed value. However, null messages need not be restricted to encoding fixed values. In fact, it is possible for a null message from j to i to refer to different values in distinct runs of the same protocol. In our protocols, there are typically processes that belong to a committee that collects initial proposals, and recommends a decision value. If a process i reports a value v to such a committee member j, then j sends i a null message to encode a recommendation of v (and an explicit message to encode recommendations of values $v' \neq v$). A null message from j to i thus dynamically encodes a value, based on their past communication.

We consider deterministic protocols that determine a process's actions based on its local state. Hence, a null message actually reports a fact about the sender's local state. In the above example, the reported fact is that the recommended value equals the value that j received from i. More generally, null messages can report arbitrary local facts and thereby serve purposes that go beyond reporting values. In the examples below, we will use null messages by a process i to report facts such as "*i has decided*," and "*i has received a unanimous recommendation*."

1.1 Global Validation via Silent Rounds

Going beyond pairwise communication, coordinated use of null messages by many processes gives rise to a powerful technique. Recall that when a correct process j receives a message reporting

This work was funded in part by ISF grant 2061/19. Guy Goren's was partly supported by a grant from the Technion Hiroshi Fujiwara cyber security research center and the Israel cyber bureau, as well as by a Jacobs fellowship. Yoram Moses is the Israel Pollak academic chair at the Technion.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '20, Virtual Event, Italy
© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.
978-1-4503-7582-5/20/08…$15.00
DOI: 10.1145/3382734.3405717

---

[1] This is no longer the case when signatures are allowed [6]. A single signed message (or a signed statement within a message) can reliably convey information that a null message cannot.

that some fact φ holds from a process i in the Byzantine model, it can infer that *if i is correct then φ holds*. It follows that, more generally, if j receives messages reporting φ_i from all processes $i \in \mathbb{P}$ then j learns that the φ_i's of all correct processes i are true. Although j's uncertainty regarding the identity of the correct processes may cause it to be unsure which of these facts holds, the information obtained in this manner can still be valuable, since the success of a protocol in the Byzantine model is typically defined in terms of events that take place at correct processes (after all, the protocol does not control the faulty processes). In Consensus, for example, the three conditions of Decision, Agreement and Validity are all stated in terms of the correct processes. Thus, for example, a process may be able to terminate in Consensus once it discovers that every correct process has all of the information it needs in order to decide. A silent round, involving no communication, will serve us as a communication-efficient tool for detecting global system properties of this type, and will be especially useful in failure-free runs (in which all processes in $\mathbb{P}$ are correct). We proceed as follows.

Definition 1.1. For every process i, let φ_i be a fact regarding i's local state. Denote $\bar{\varphi}_c \triangleq \bigwedge_{\text{correct } i} \varphi_i$, and fix some time $m \geq 0$. A protocol $\mathcal{P}$ is said to implement a **silent validation round for** $\bar{\varphi}_c$ (svr $(\bar{\varphi}_c)$ for short) in round $m{+}1$ if in every run of the protocol, each correct i sends messages to everyone in round $m+1$ in case φ_i does not hold at time m, and sends no message to anyone in round $m+1$ if φ_i does hold.

The fact $\bar{\varphi}_c$ states that for every correct process i (in the current run) the fact φ_i holds. Clearly, its truth depends on the identity of the correct processes which, in turn, is determined per run by the adversary. By design, silent validation rounds satisfy the following:

Theorem 1.2. *Assume that $\mathcal{P}$ implements an svr$(\bar{\varphi}_c)$ in round $m+1$, and fix a run r of $\mathcal{P}$. A process j that receives no messages whatsoever in round $m+1$ knows at time $m+1$ that $\bar{\varphi}_c$ was true at time m.*

We shall refer to properties of the form $\bar{\varphi}_c$ as *global milestones*. Knowing that a global milestone has been reached is often valuable in distributed protocols. For example, in many popular Consensus protocols processes maintain a local estimate of the decision value, and once the estimates of all correct processes are the same, the decision value is determined. The fact that all estimates are the same corresponds to a global milestone. As we will see later on, this global milestone can be detected by all correct processes, following a properly designed silent validation round. Existing protocols in the literature often employ other means to detect global milestones. Indeed, a variety of fault-tolerant protocols use long silent phases consisting of more than t rounds to verify that a specific milestone has been reached (e.g., [1, 9, 10]). The time complexity of these protocols in the common case of failure-free runs can easily be reduced if the long multi-round phase is replaced by a single svr.

While silent validation rounds have not been defined explicitly before, they have implicitly appeared in several distributed protocols in the literature. One interesting application of this technique is in broadcast-based protocols for radio networks, where such rounds are used to overcome possible malicious behavior (see, e.g., [4, 7]). Moreover, the Atomic Commitment protocols in [8] use silent validation rounds to gain communication efficiency. We remark that although silent validation rounds can sometimes convey the global information $\bar{\varphi}_c$ practically for free, it is clear that they might induce high costs when $\bar{\varphi}_c$ does not hold. As discussed in [8], this is inherent in using null messages effectively. At best, such use will shift costs among executions; it does not eliminate them totally. When we optimize Consensus for the common case of failure-free runs in Section 2, silent validation will allow us to reduce the communication costs considerably in the common case.

2 IMPROVING BYZANTINE CONSENSUS PROTOCOLS

In this section we use the insights from Section 1 to design novel Consensus protocols for the Byzantine model. Independent of the methodology, our goal is to obtain protocols that are fast and communication efficient in the common case. In fact, we provide a general transformation that optimizes the common-case behavior of existing protocols. We use a modular approach, in which the protocol designer is free to choose a Byzantine Consensus protocol of her liking (which we refer to as the *base protocol*). If no failures occur, a very short and efficient protocol called a *common-case layer* is executed and Consensus is achieved with relative ease. Execution reverts to the base protocol if failures prevent the common-case layer from reaching Consensus quickly. This affords the designer with the best of both worlds—excellent behavior in the common case, and execution of a base protocol with the properties that the designer favors most when failures do occur. Crucially, when failures occur, executing the common-case layer adds a negligible cost to runs of the base protocol. Turpin and Coan used a similar layer to convert binary Consensus protocols into multi-valued ones [13]. We remark that our solutions have the additional quality that they emulate a fair voting mechanism in the common case. I.e., they decide on the majority values in binary Consensus, and on a plurality value in multi-valued Consensus.

We now define a layer, called SmallCouncil, which optimizes the common-case behavior of any chosen Consensus protocol. In failure-free runs, any protocol starting with this layer uses fewer messages (and bits) than any previously known protocol, while it decides fast – at time 3. This allows the protocol designer to emulate majority voting in failure-free runs with the best known communication costs while incurring negligible costs on the base protocol in runs with failures.

Roughly speaking, SmallCouncil (whose pseudocode is presented in Fig. 1) works as follows: A committee of $t+1$ processes is *a priori* defined (we call it the *Council*). In the first round all processes report their initial proposals to the Council. Each Council member then calculates the majority value among the reported votes, and sends a recommendation to all processes accordingly. At time 2, a process that received a unanimous recommendation from the Council, sets its estimation to be that recommendation and remains silent in the third round. Otherwise (if it receives conflicting recommendations), it sets its estimation to be its initial proposal and broadcasts an $\overline{err}$ message to alert all of the problem. A process that receives no $\overline{err}$ message in the third round, decides at time 3 on its estimation. If it does receive a third round $\overline{err}$ message, it broadcasts a *¡help!* message to alert everyone that it does not know what to decide. Any process that receives a *¡help!* message in the fourth round will participate in an instance of the base protocol, starting from time 4. The process uses its estimation value as a proposal in the base protocol.

In terms of communication, in this layer, both the third and fourth rounds serve as silent validation rounds (Definition 1.1). The third is an svr for the global fact "*every correct process have received a unanimous recommendation,*" and the fourth for "*all correct processes have decided.*"

Theorem 2.1. *Let $k \geq 3$ and let* `Base.Protocol` *be a binary Consensus protocol for $n > kt$. Then composing* SmallCouncil *onto* `Base.Protocol` *yields a binary Consensus protocol in which*

1. *In failure-free runs, decisions occur after 3 rounds and at most $n(t + 1.5)$ bits are communicated, while*
2. *When failures occur, at most $n(t + 1.5) + 2n^2$ bits are sent by correct processes, and 4 rounds elapse before reverting to the base protocol.*
3. *In failure-free runs, the composed protocol decides on the majority value.*

```
time 0  $\forall i \in \mathbb{P}$:
    $\forall j \in \{0, 1, 2, ..., t\}$ do
        if $j$ mod 2 = $v_i$ then be silent
        else send $v_i$ to $j$
time 1  $\forall j \in \{0, 1, 2, ..., t\}$:
    values$_j[i] \leftarrow \left\{ \begin{array}{ll} j \bmod 2 & \text{if no message from } i \text{ was received} \\ (j+1) \bmod 2 & \text{otherwise} \end{array} \right\}$
    $rec_j \leftarrow MAJ($values$_j)$
    $\forall i \in \mathbb{P}$
        if values$_j[i] = rec_j$ then be silent
        else send $rec_j$ to $i$
time 2  $\forall i \in \mathbb{P}$:
    if received a unanimous recommendation $rec$ from $\{0, 1, ..., t\}$
        then $est_i \leftarrow rec$
    else $est_i \leftarrow v_i$; send '$\overline{err}$' to all;
time 3  $\forall i \in \mathbb{P}$:
    if did not receive any '$\overline{err}$' then decide($est_i$)
    else send '¡help!' to all
time 4 and beyond  $\forall i \in \mathbb{P}$:
    if did not receive any '¡help!' then halt
    else    dec $\leftarrow$ Base.Protocol($est_i$)
            if undecided after time 3 then decide(dec)
```

Figure 1: SmallCouncil **– A layer transforming any byzantine Consensus protocol to be the most communication efficient known in the common case.**

The bit complexity of the SmallCouncil layer is 4 times the best-known lower bound of $\Omega(nt/4)$ bits for this case from [5, 10]. The best previously-known communication behavior is by the Early Stopping Phase King protocol of [2], which requires up to $8n^2$ bits and takes up to 8 rounds to decide in failure-free runs. Our SmallCouncil layer achieves a 24-fold improvement in bit complexity, while also reducing the decision time (from 8 to 3 rounds). Moreover, protocols such as Phase King are far from emulating majority in the common case. Indeed, even in failure-free executions of Phase King the King's proposal typically wins. Finally, when failures do occur, SmallCouncil adds only 4 rounds and sends fewer than $3n^2$ bits. Consequently, prepending SmallCouncil does not change the asymptotic complexity of any possible base protocol.

3 DISCUSSION

Whereas in asynchronous systems information flows only via message chains [3, 11], synchronous systems allow for more complex information flow mechanisms. In particular, null messages can be employed in synchronous settings and are useless in asynchronous ones. While null messages were traditionally used mainly for detecting failures, Lamport [12] suggested that they can be used to do much more. Moreover, since sending a message incurs significant overhead beyond the number of bits of data contained in the message, null messages should ideally be used wherever possible.

Null messages can be used in more subtle ways than encoding an *a priori* fixed value. For example, in the second round of our layer we use "on the fly" encoding of null messages from a Council member j to a process i based on the message that i sent j in the first round. Notice that the first message influences the meaning of the null message in the other direction. In this manner a single (explicit) message from i to j induces bidirectional information flow. This phenomenon is unique to synchronous systems.

Recall that the null-message techniques developed in this paper, and in particular the svr primitive, were put to work in the design of Consensus protocols that are very efficient in common case executions. The result is general transformations that convert any protocol into one that guarantees excellent common-case behavior at essentially no additional costs in the worst case. SmallCouncil improves the previously known upper bound by a factor of 24 to $\sim nt$ messages and bits. Note that there is a known lower bound of $\sim nt/4$ explicit messages for achieving Consensus in failure-free runs [5, 10]. A factor of 4 gap between the lower and upper bounds remains; closing that gap is an open question.

REFERENCES

[1] Eugene S Amdur, Samuel M Weber, and Vassos Hadzilacos. 1992. On the message complexity of binary byzantine agreement under crash failures. *Distributed Computing* 5, 4 (1992), 175–186.
[2] Piotr Berman, Juan A Garay, and Kenneth J Perry. 1992. Optimal early stopping in distributed consensus. In *International Workshop on Distributed Algorithms*. Springer, 221–237.
[3] KM Chandy and Jayadev Misra. 1986. How processes learn. *Distributed Computing* 1, 1 (1986), 40–52.
[4] Gregory Chockler, Murat Demirbas, Seth Gilbert, Nancy Lynch, Calvin Newport, and Tina Nolte. 2008. Consensus and collision detectors in radio networks. *Distributed Computing* 21, 1 (2008), 55–84.
[5] Danny Dolev and Rüdiger Reischuk. 1985. Bounds on information exchange for Byzantine agreement. *Journal of the ACM (JACM)* 32, 1 (1985), 191–204.
[6] Danny Dolev and H. Raymond Strong. 1983. Authenticated algorithms for Byzantine agreement. *SIAM J. Comput.* 12, 4 (1983), 656–666.
[7] Seth Gilbert, Rachid Guerraoui, and Calvin Newport. 2009. Of malicious motes and suspicious sensors: On the efficiency of malicious interference in wireless networks. *Theoretical Computer Science* 410, 6-7 (2009), 546–569.
[8] Guy Goren and Yoram Moses. 2018. Silence. In *Proceedings of the 2018 ACM Symposium on Principles of Distributed Computing (PODC '18)*. ACM, New York, NY, USA, 285–294. https://doi.org/10.1145/3212734.3212768
[9] Rachid Guerraoui and Jingjing Wang. 2017. How fast can a distributed transaction commit?. In *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. ACM, 107–122.
[10] Vassos Hadzilacos and Joseph Y Halpern. 1993. Message-optimal protocols for Byzantine agreement. *Mathematical Systems Theory* 26, 1 (1993), 41–102.
[11] Leslie Lamport. 1978. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications* (1978).
[12] Leslie Lamport. 1984. Using time instead of timeout for fault-tolerant distributed systems. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 6, 2 (1984), 254–280.
[13] Russell Turpin and Brian A Coan. 1984. Extending binary Byzantine agreement to multivalued Byzantine agreement. *Inform. Process. Lett.* 18, 2 (1984), 73–76.